

Project Report

Blood Donation System

Submitted in partial fulfillment of the requirements for the project work

Project Type: Web Based Blood Donation Management System

Submitted By

Name: _____

Registration No: _____

Department: _____

Submitted To

Lecturer/Supervisor: _____

Institution: _____

Academic Year: 2026

Certificate

This is to certify that the project report titled "Blood Donation System" is a bonafide work carried out as part of the academic project requirement. The system has been developed using React, Express.js, Node.js and MySQL to support donor registration, blood request handling, donor matching, inventory tracking and support communication.

The report explains the analysis, design, implementation, testing and conclusion of the project. The work presented in this document is based on the project files and implementation available in the Blood Donation System application.

Project Guide / Supervisor

Head of Department

Declaration

I hereby declare that this project report entitled "Blood Donation System" has been prepared by me based on the software project developed for managing blood donation related activities. The information, diagrams and explanations included in this report are written for academic documentation purposes.

The project aims to provide a simple and organized platform where users can request blood, donors can manage donation activity, and administrators can coordinate donors, requests, inventory and support messages.

Student Signature

Date

Acknowledgement

I would like to express my sincere gratitude to my project guide, teachers and institution for providing guidance and encouragement throughout the development of this project. Their support helped in understanding the real-life importance of blood donation systems and in converting that idea into a working web application.

I also thank my friends and family members for their motivation during the project development period. This project improved my practical knowledge of frontend development, backend API creation, database handling and project documentation.

Abstract

Blood donation is a critical healthcare support activity. In emergency situations, patients and hospitals often need to find suitable blood donors quickly. Manual donor records, phone-based coordination and unorganized request tracking can delay this process. The Blood Donation System is developed to reduce these difficulties by providing a web-based platform for donors, users and administrators.

The system allows users to register, login, request blood, track request status and search for donors. Donors can update their profile, view assigned requests, respond to notifications and check donation activity. Administrators can manage donor records, approve and monitor blood requests, notify matching donors, maintain blood inventory, view expiry alerts, list user accounts and handle chat support conversations.

The project uses React for the frontend interface, Express.js and Node.js for backend APIs, and MySQL for persistent data storage. Passwords are stored using salted hashes. The system also includes validation, role-based navigation and automatic expiry status handling for blood inventory.

Table of Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

Chapter 1 - Introduction 1

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope of the Project 3

Chapter 2 - Literature Review and Existing System 4

Chapter 3 - System Analysis 7

Chapter 4 - System Design 12

Chapter 5 - Implementation 18

Chapter 6 - Testing 25

Chapter 7 - Conclusion and Future Enhancement 29

References 31

Appendix 32

Chapter 1

Introduction

1.1 Introduction

Blood is an essential medical resource required during surgeries, accidents, childbirth complications, cancer treatments and many emergency cases. A successful blood donation process depends on the availability of donors, accurate donor details, quick communication and proper tracking of requests.

The Blood Donation System is a web application named Blood Bridge. It connects patients, normal users, donors, hospitals and administrators through a single organized platform. The application provides public pages, login and registration, donor search, blood request forms, admin dashboards and support chat features.

1.2 Problem Statement

In many situations, blood donor information is maintained manually or through scattered contact lists. This creates problems such as outdated donor records, delayed donor matching, lack of request status visibility, poor inventory tracking and difficulty in contacting the responsible administrator. These problems become serious during emergency blood requirements.

1.3 Objectives

- To develop a web-based platform for managing donors, users and administrators.
- To allow users and donors to submit normal or emergency blood requests.
- To help administrators match requests with suitable donors based on blood group.
- To maintain blood inventory with units, hospitals, status and expiry dates.
- To provide notifications for donor responses and inventory expiry alerts.
- To provide chat support between users/donors and administrators.

1.4 Scope of the Project

The scope of this project includes user registration, donor registration, role-based login, donor search, blood request submission, admin request management, donor notification handling, blood inventory maintenance, expiry alerts, profile updates, password changes and support conversations.

1.5 Technologies Used

Layer Technology Purpose

Frontend React, React Icons, CSS User interface and dashboards

Backend Node.js, Express.js REST API and business logic

Database MySQL Persistent storage

Security Crypto PBKDF2 hashing Password hashing with salt

Configuration dotenv Environment based settings

Chapter 2

Literature Review and Existing System

2.1 Literature Review

Healthcare management systems are designed to improve speed, accuracy and reliability in medical services. Blood donation management is one of the important areas where digital systems can reduce delays. A computerized system can store donor details, search donors by blood group and location, track requests and notify responsible users.

Modern web applications usually use a frontend framework, backend API and relational database. This separation improves maintainability. React helps create reusable components and dashboards, Express.js supports API endpoints and MySQL provides structured storage for donor, request, inventory and account records.

2.2 Existing System

In a manual blood donation process, donor details may be stored in notebooks, spreadsheets or phone contacts. When a request arrives, the administrator must manually search for donors and call them individually. Request status may not be visible to the requester, and inventory expiry may be missed if records are not checked regularly.

2.3 Disadvantages of Existing System

- Donor records can become outdated or duplicated.
- Searching donors manually takes more time.
- Users cannot easily track their request status.
- Inventory expiry dates are difficult to monitor manually.
- Communication between donor, user and admin is not centralized.
- Manual records are less secure and more difficult to back up.

2.4 Proposed System

The proposed Blood Donation System provides an online interface for all major roles. Users can request blood and search donors. Donors can manage profiles and respond to assigned requests. Administrators can create donor accounts, approve requests, notify matching donors, update request progress, maintain blood stock and communicate through chat support.

2.5 Advantages of Proposed System

- Centralized MySQL database for donor, user, request and inventory data.
- Role-based dashboards for admin, donor and user.
- Faster donor matching using blood group and donor records.
- Inventory expiry alerts for expired or soon-expiring stock.
- Secure password storage using salted hash values.
- Support chat conversation history for user assistance.

Chapter 3

System Analysis

3.1 Requirement Analysis

Requirement analysis identifies what the system must perform and what conditions are required for successful operation. The Blood Donation System has three main user roles: admin, donor and user. Each role has separate permissions and dashboard actions.

3.2 Functional Requirements

Module Functional Requirement

Authentication Register users/donors, login accounts, reset password and change password.

User Module Create blood requests, view request status, find donors, update profile and use support chat.

Donor Module Maintain donor profile, view notifications, respond to requests and check donation activity.

Admin Module Manage donors, requests, users, blood inventory, expiry alerts and support inbox.

Inventory Store blood group, units, hospital, expiry date and stock status.

Chat Support Create conversations, list messages, reply and close conversations.

3.3 Non Functional Requirements

- Usability: The interface should be simple for patients, donors and admins.
- Reliability: Data should be stored persistently in MySQL.
- Security: Passwords should not be stored in plain text.
- Maintainability: Frontend pages, components and backend routes should be separated.
- Performance: Common data should load through API calls and refresh where needed.
- Scalability: The structure should support adding more hospitals or notification channels later.

3.4 Hardware Requirements

Component Minimum Requirement

Processor Intel Core i3 or equivalent

RAM 4 GB minimum, 8 GB recommended

Storage 500 MB free space for project and dependencies

Network Local network or internet connection for deployment

3.5 Software Requirements

Software Use

Node.js and npm Run frontend and backend project dependencies

MySQL Server Database server

Web Browser Access React application

Code Editor Edit and maintain project files

3.6 Feasibility Study

The project is technically feasible because it uses common open-source web technologies. It is operationally feasible because the modules match real blood donation workflows. It is economically feasible because no expensive proprietary tools are required for development.

Chapter 4

System Design

4.1 System Architecture

The system follows a client-server architecture. The React frontend sends requests to the Express backend. The backend validates input, applies business rules and communicates with the MySQL database. API responses are returned as JSON and shown in the user interface.

React Frontend

Pages and Components

Express API

Routes and Validation

Business Logic

Matching and Alerts

MySQL Database

Persistent Records

4.2 User Roles

Role Main Responsibilities

Admin Manage donors, requests, inventory, users, alerts and chat support.

Donor Maintain donor profile, view assigned requests and respond to notifications.

User Request blood, track own requests, search donors and contact support.

Guest View public pages, register, login and start limited support conversation.

4.3 Data Flow Diagram

User/Donor

Submit Request

Admin

Reviews Request

Matching Donor

Notified

Requester

Receives Status

4.4 Database Design

The backend automatically creates the required database and tables when the server starts. The main tables are donors, blood_requests, users, blood_inventory, chat_conversations and chat_messages.

Table Purpose Important Fields

users Stores login accounts fullName, username, phone, bloodGroup, role, passwordHash, status

donors Stores searchable donor profiles name, phone, bloodGroup, location, birthday, lastDonation

blood_requests Stores request workflow requester, bloodGroup, units, hospital, type, status, donorResponse

blood_inventory Stores blood stock bloodGroup, units, hospital, expiryDate, status

chat_conversations Stores support threads userName, userRole, subject, status,
lastMessageAt

chat_messages Stores chat messages conversationId, senderName, senderRole, message

4.5 Main Screens

- Home page with donor search and blood request actions.
- Login and registration pages with user and donor account creation.
- User dashboard for requests, donor search, profile and support.
- Donor dashboard for profile, donations, requests and notifications.
- Admin dashboard for donor, request, inventory, user and support management.

Chapter 5

Implementation

5.1 Project Structure

Path Description

src/App.js Main application controller and role-based navigation guard.

src/pages Full screen pages for public, auth, user, donor and admin modules.

src/components Reusable UI components such as forms, layout, metric cards and status badges.

src/services/api.js Shared API helper for frontend requests.

src/data/navigation.js Sidebar menu definitions for admin, donor and user.

server.js Express backend, database setup, API routes and business logic.

5.2 Frontend Implementation

The frontend is implemented with React. The main App component stores the active screen and current logged-in user. Protected screens such as admin, donor, user, request, donors and profile are guarded. Guests are redirected to login before opening protected pages.

The user interface is separated into pages and reusable components. DashboardLayout provides sidebar navigation for role-based dashboards. StatusBadge displays status values such as Pending, Approved, Accepted, Rejected, Available and Expired.

5.3 Backend Implementation

The backend uses Express.js. It loads database configuration from environment variables and creates a MySQL connection pool. On startup, it creates missing database tables and ensures that a default admin account is available.

Main API groups:

/api/auth/register

/api/auth/login

/api/auth/change-password

/api/donors

/api/requests

/api/inventory

/api/users

/api/admins

/api/chat/conversations

5.4 Authentication and Security

User passwords are not returned to the frontend. The backend stores passwordHash and passwordSalt values generated with PBKDF2. The sanitizeUser function removes password related fields before sending account data to the client.

5.5 Donor Management Module

Administrators can add donor records with name, phone, blood group, location, birthday, last donation, username and password. The system also includes a donor

credential generator based on donor name and birthday. Donor records are searchable using ID, name, phone, blood group and location.

5.6 Blood Request Module

Users, donors and admins can create blood requests. Request fields include requester, blood group, units, hospital, type and status. Admins can analyse requests, find matching donors, send notification status and update donor response status.

5.7 Blood Inventory and Expiry Alerts

The inventory module records blood group, unit count, hospital, expiry date and status. The backend checks expiry dates and marks expired items. The admin dashboard shows expiry alerts for stock that is expired or close to expiry.

5.8 Chat Support Module

The chat module allows guests, users and donors to start support conversations. Administrators can view support inbox messages, reply to conversations and close conversations. Chat messages are stored in the database with sender details and timestamps.

Chapter 6

Testing

6.1 Testing Approach

Testing was planned around major workflows. Each module was checked by entering valid and invalid data, submitting forms, verifying database records and confirming that screens show the expected output.

6.2 Test Cases

No Test Case Expected Result Status

1 User registration with required details Account is created and stored in users table

Pass

2 Login with valid username and password User is redirected to correct role dashboard

Pass

3 Login with wrong password Error message is returned Pass

4 Add donor from admin dashboard Donor record appears in donor list Pass

5 Search donors by blood group Matching donor records are displayed Pass

6 Create blood request Request is saved and shown in request records Pass

7 Notify matching donor Notification status changes to Sent Pass

8 Donor accepts or rejects request Donor response is updated Pass

9 Add expired inventory date Status becomes Expired Pass

10 Start chat support conversation Conversation and first message are stored Pass

6.3 Validation Testing

Forms require important fields such as full name, username, password, donor phone, blood group, location, hospital and expiry date. Phone number validation checks that phone numbers contain exactly ten digits. Password changes require a minimum length of eight characters.

6.4 Result

The tested workflows satisfy the main project requirements. The application supports role-based navigation, data storage, request tracking, donor coordination, inventory expiry handling and support messaging.

Chapter 7

Conclusion and Future Enhancement

7.1 Conclusion

The Blood Donation System successfully provides a web-based solution for organizing blood donation activities. It reduces manual record keeping by storing donors, users, requests, inventory and support conversations in a MySQL database. The role-based dashboards make the system easier to use for administrators, donors and normal users.

The project demonstrates practical use of React, Express.js, Node.js and MySQL. It also includes important application features such as password hashing, profile management, donor matching, request notifications and expiry alerts.

7.2 Limitations

- The system currently uses dashboard notifications instead of SMS or email delivery.
- Hospital accounts are not implemented as a separate role.
- Location matching is text-based and does not use maps or distance calculation.
- Advanced analytics and printable admin reports are not fully automated.

7.3 Future Enhancements

- Add SMS and email notifications for urgent blood requests.
- Add hospital role with hospital-specific request and stock management.
- Integrate map-based donor search using location distance.
- Add PDF report generation for donor lists, request history and inventory reports.
- Add appointment scheduling for blood donation campaigns.
- Deploy the project to a cloud server for real-world access.

References

- React Documentation - Component based frontend development.
- Express.js Documentation - Web application framework for Node.js.
- MySQL Documentation - Relational database management system.
- Node.js Documentation - JavaScript runtime environment.
- Project source files: Blood Donation System, local workspace implementation.

Appendix

Appendix A: Environment Variables

MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=your_mysql_password
MYSQL_DATABASE=blood_donation_system
ADMIN_NAME=Blood Bridge Admin
ADMIN_USERNAME=admin
ADMIN_PASSWORD=admin123
PORT=5000

Appendix B: Run Commands

npm install
npm run server
npm start
npm run build

Appendix C: Important Project Files

File Purpose

server.js Backend API, MySQL schema setup and route handling
src/App.js Frontend navigation and role guard
src/pages/admin/AdminDashboard.jsx Admin management screens
src/pages/donor/DonorDashboard.jsx Donor dashboard screens
src/pages/user/UserDashboard.jsx User dashboard screens
src/data/navigation.js Admin, donor and user menu items